

中山大学计算机学院

人工智能

本科生实验报告

课程名称: Artificial Intelligence

学号	22336203	姓名	沈苇杭
----	----------	----	-----

一、实验题目

在 C artPole-v0 环境中实现 DQN 算法

二、实验内容

1. 算法原理

- 1) Q-Learning 算法: 将 state 和 action 构建一张 Q-table 表存储 Q ($Q(s,a)$, 指在某一个时刻的 state 下采取动作 action 能够获得收益的期望) 值, 然后根据 Q 值选取能够获得最大收益的动作。
- 2) Native DQN: 在每一个回合中, 在每个时间步, 智能体根据贪心策略和所有动作的值函数选择对应的动作, 环境状态转移, 返回对应的奖励值, 并更新神经网络参数。
- 3) 经验回放: 将智能体探索环境得到的数据储存起来, 然后随机采样小批次样本更新深度神经网络的参数, 增加数据利用率, 减少连续样本相关性。
- 4) 目标网络: 额外引入一个目标网络 (和 Q 网络具有同样的网络结构), 此目标网络不更新梯度, 每隔一段时间将 Q 网络的参数赋值给此目标网络, 提高算法稳定性。

2. 关键代码展示

```
1) class AgentDQN(Agent):
    def __init__(self, env, args):
        super(AgentDQN, self).__init__(env)
        self.env = env
        self.args = args
        # 选择 GPU 或者 CPU
        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        # 创建网络
        self.q_network = QNetwork(4, 256, 2).to(self.device)
        # 创建优化器, 学习率 0.001
        self.optimizer = optim.Adam(self.q_network.parameters(), lr=0.001)
        # 创建目标网络
        self.target_network = copy.deepcopy(self.q_network)
```

```

# 创建一个容量 4000 的缓冲区
self.replay_buffer = ReplayBuffer(5000)

# 缓冲区最小大小为 400
self.minimal_size = 500

# 批次大小
self.batch_size = 50

# 折扣因子
self.gamma = 0.9

# 探索率
self.epsilon = 0.9

# 探索率的衰减率
self.epsilon_decay = 0.95

# 最小探索率
self.epsilon_min = 0.001

# 目标网络更新频率
self.update_target_every = 10

# 步数
self.steps = 0

return

```

在类的构造函数中，分别初始化神经网络、缓冲区和 Q-learning 算法相关的参数。创建 Q-network（input 是 state，output 是 action），目标网络，优化器。创建缓冲区并设置最低大小。设置批次大小、折扣因子、探索率、探索率的衰减率、最小探索率和目标网络更新频率，初始化步数。

```

2) def run(self):
    # 每个 episode 的总 reward
    total_rewards = []
    for episode in range(100):
        # 重置 state 为初始状态
        state = self.env.reset()
        # 取出实际的状态值
        state = state[0]
        # 初始化奖励为 0
        total_reward = 0
        # 初始化完成标志为 false
        done = False
        # 未完成
        while not done:

```

```

        # 训练模式，根据当前状态选择动作
        action = self.make_action(state, test=False)
        # 执行动作，获得下一个状态，奖励，完成标志
        next_state, reward, terminated, truncated, _ = self.env.step(action)
        done = terminated or truncated
        # 把信息放进缓冲区
        self.replay_buffer.push(state, action, reward, next_state, done)
        # state 转移
        state = next_state
        # 计算 reward
        total_reward += reward
        # 训练神经网络
        self.train()
        self.steps += 1

    # 更新探索率
    self.epsilon = max(self.epsilon_min, self.epsilon * self.epsilon_decay)
    # 添加 reward
    total_rewards.append(total_reward)
    # 打印信息
    print("Episode: ", episode + 1, "Total Reward: ", total_reward, "Epsilon: ",
          self.epsilon)
    return total_rewards

```

最外层的 run () 函数：运行 100 个 Episode。在每个 Episode 中，如果完成标志为 False，即未完成，就进行训练。首先调用 make_action() 函数，选择动作，并获得动作执行后的下一个 state、奖励、完成标志。这里的完成标志包括 Terminated 和 Truncated 两种情况，其一完成即为完成。将当前 state 等相关信息放入缓冲区，用于后续训练。随后，state 转移，合计 reward，开始训练 q-network。每完成一个 Episode，要更新探索率。

```

3) def make_action(self, observation, test=True):
    # 随机数>探索率
    if random.random() > self.epsilon:
        # 预测动作
        observation = torch.tensor(observation,
                                   dtype=torch.float32).unsqueeze(0).to(self.device)
        with torch.no_grad():
            # 最大 q 值动作
            action = self.q_network(observation).argmax().item()
    # 随机选择一个动作
    else:
        action = self.env.action_space.sample()
    return action

```

产生下一个动作，根据产生的随机数和探索率的关系，有根据 q 值产生动作和随机产生动作两种情况。

```

4) def train(self):
    # 如果缓冲区长度小于最小长度，不训练
    if len(self.replay_buffer) < self.minimal_size:
        return
    # 从缓冲区中抽样，获得抽样结果和这些结果的 index
    transitions, indices = self.replay_buffer.sample(self.batch_size)
    # 将抽样结果分类组成元组，再放进一个列表，方便处理
    batch = list(zip(*transitions))
    # 第一类，状态
    states = torch.tensor(np.array(batch[0]), dtype=torch.float32).to(self.device)
    # 第二类，行动
    actions = torch.tensor(batch[1], dtype=torch.int64).to(self.device)
    # 第三类，奖励
    rewards = torch.tensor(batch[2], dtype=torch.float32).to(self.device)
    # 第四类，下一个状态
    next_states = torch.tensor(np.array(batch[3]),
    dtype=torch.float32).to(self.device)
    # 第五类，完成标志
    dones = torch.tensor(batch[4], dtype=torch.float32).to(self.device)
    # 计算当前 state 下选择的 action 对应的 q 值
    q_values = self.q_network(states).gather(1, actions.unsqueeze(1)).squeeze(1)
    # next_state 下目标网络计算的最大 q 值
    next_q_values = self.target_network(next_states).max(1)[0]
    # 计算预期的 q 值，如果 dones 为真，那么 q 值是 reward，否则要折现
    expected_q_values = rewards + (self.gamma * next_q_values * (1 - dones))
    # 均方误差计算 loss
    loss = nn.MSELoss()(q_values, expected_q_values.detach())
    # 更新参数
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()
    # 更新目标网络
    if self.steps % self.update_target_every == 0:
        self.target_network.load_state_dict(self.q_network.state_dict())
    # 更新缓冲区的优先级
    self.replay_buffer.update_priorities(indices, (q_values -
    expected_q_values).abs().cpu().detach().squeeze(-1).numpy())
    return

```

训练 q-network。从缓冲区中抽样，分类，计算 q 值，计算 loss，更新参数。

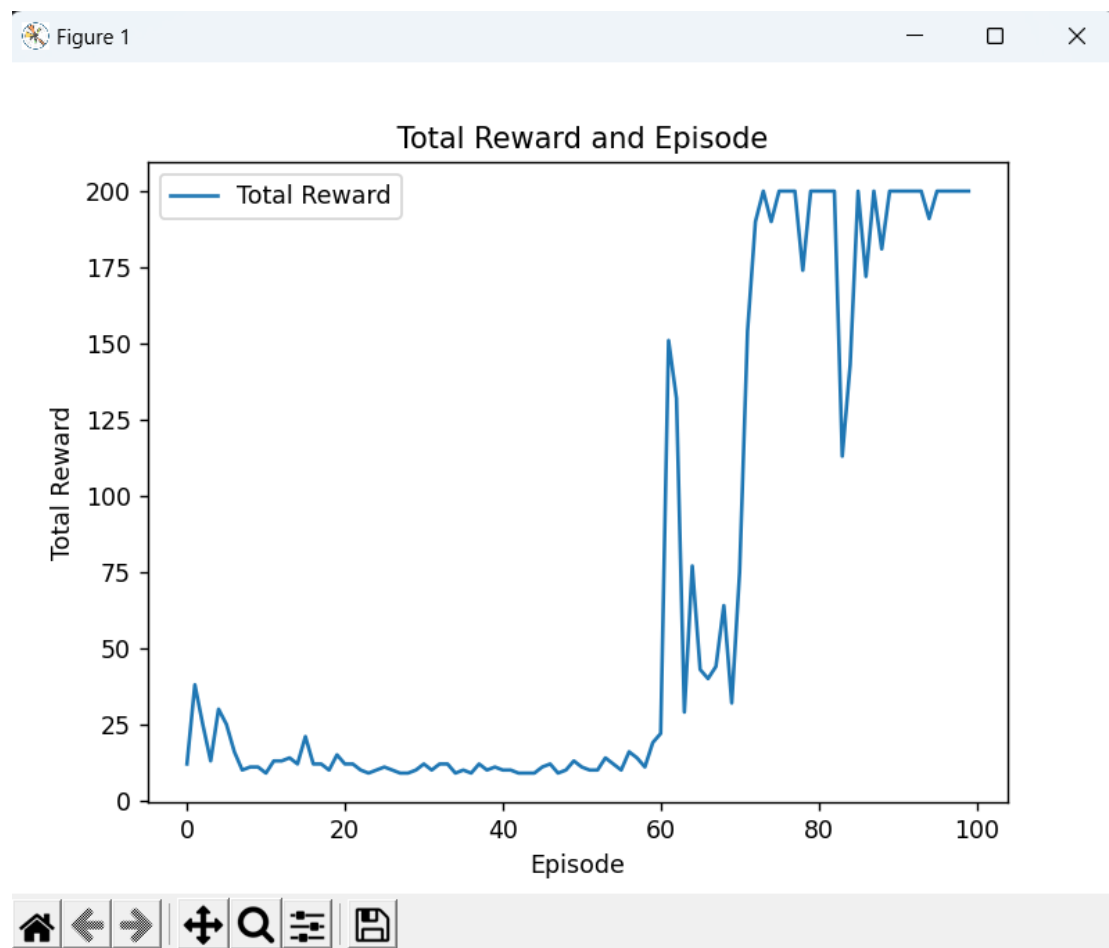
三、 实验结果及分析

1. 实验结果展示示例

```
Episode: 93 Total Reward: 200.0 Epsilon: 0.007630233023649456
Episode: 94 Total Reward: 200.0 Epsilon: 0.007248721372466983
Episode: 95 Total Reward: 191.0 Epsilon: 0.006886285303843633
Episode: 96 Total Reward: 200.0 Epsilon: 0.0065419710386514516
Episode: 97 Total Reward: 200.0 Epsilon: 0.0062148724867188785
Episode: 98 Total Reward: 200.0 Epsilon: 0.005904128862382934
Episode: 99 Total Reward: 200.0 Epsilon: 0.005608922419263787
Episode: 100 Total Reward: 200.0 Epsilon: 0.005328476298300597
```

如图，这是最后几个 Episode 的 reward 和 Epsilon

2. 评测指标展示及分析



如图，Total Reward 最终收敛在 200 附近。

四、参考资料

1. [强化学习笔记：Gym 入门--从安装到第一个完整的代码示例 gym 安装-CSDN 博客](#)
2. [强化学习 7——一文读懂 Deep Q-Learning \(DQN\) 算法_deep q learning-CSDN 博客](#)
3. [深度 Q 网络 \(deep Q network, DQN\) 原理&实现 - 缙云山车神 - 博客园 \(cnblogs.com\)](#)
4. [【机器学习】强化学习 \(六\) -DQN\(Deep Q-Learning\)训练月球着陆器示例-CSDN 博客](#)